

NODEVERSE | PRODUCT PROPOSAL

NODEVERSE

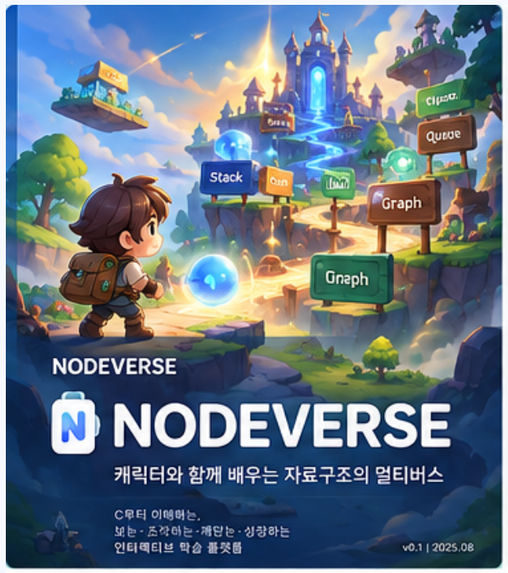
자료구조 학습 멀티버스

아바타 시뮬레이션 × C 언어 ×
학습 행동 분석 × AI 개선·배포

- React Native / Expo 기반 모바일 학습 앱
- Kafka · Spark · MongoDB 학습 데이터 플랫폼
- 관리자 채팅 승인 → AI 코드 수정 → 테스트 → 배포

전체 서비스 콘셉트 · 두 번째 기획 디자인 시안

학습 앱 · 자료구조 학습 시나리오 · 아키텍처 · 관리자 대시보드를 종합한 초기 비주얼 보드



1. 서비스 개요 01

NODEVERSE는 아바타 캐릭터가 직접 자료구조를 실행하는 과정을 보여주고, 사용자가 조작하여 C 코드와 함께 학습하는 모바일 기반 인터랙티브 학습 플랫폼입니다.

- 시각적 이해
- 직접 조작
- C 코드 연동
- 개인 맞춤 학습
- 데이터 기반 개선
- AI 콘텐츠 진화

핵심 가치

- 추상적인 자료구조를 직관적으로 시각화
- 캐릭터 기반 인터랙션으로 학습 몰입도 향상
- C 언어와 메모리 구조까지 함께 이해
- 사용자 데이터 기반 개인화 학습 제공
- AI가 지속적으로 콘텐츠를 개선하는 학습 생태계

2. 주요 특징 02

1 캐릭터가 직접 실행
캐릭터가 공(데이터)을 옮기거나, 노드를 연결하는 등 자료구조 동작을 직접 수행합니다.

2 C 코드와 동기화
캐릭터 행동과 C 코드, 메모리 구조 변화를 동시에 보여줍니다.

```
// 노드 삽입
newNode->next = current->next;
current->next = newNode;
```

메모리 구조 변화: 10 (0x1000) → 20 (0x2000) → 30 (0x3000)

3 직접 조작하고 실습
사용자가 캐릭터에게 명령을 내려 자료구조를 조작하고 결과를 확인합니다.

삽입 · 삭제 · 연결

4 단계별 학습 시나리오
시범 보기 → 직접 조작 → C 코드 작성 → 평가 → 복습의 단계로 구성됩니다.

시범 · 조작 · 코드 · 평가 · 학습

3. 주요 화면 시안 03



월드맵 (학습 맵) · 학습 화면 (연결 리스트 삽입) · 퀴즈 화면

4. 학습 콘텐츠 예시 04

연결 리스트 노드 삽입 과정

1 초기 상태: 10 (0x1000) → 30 (0x3000)

2 새 노드 생성: 20 (0x2000)

3 포인터 연결 (1)
`newNode->next = current->next;`
10 (0x1000) → 20 (0x2000) → 30 (0x3000)

4 포인터 연결 (2)
`current->next = newNode;`
10 (0x1000) → 20 (0x2000) → 30 (0x3000)

5. 캐릭터 & 세계관 05

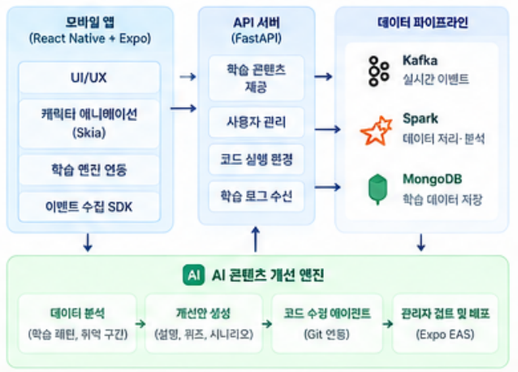
자료구조의 개념을 탐험하는 멀티버스 세계관



- 노드 (Node) — 메인 캐릭터 학습자의 아바타
- 비트 (Bit) — 데이터를 상징하는 동반자
- 프로세서 C — C 언어와 메모리 구조를 설명하는 가이드
- 버그몬 — 학습 도전 과제를 제공하는 존재

- 연결 리스트 힐즈 — 연결과 참조의 세계
- 스택 타워 — 쌓고 꺼내는 구조
- 트리 포레스트 — 계층적 탐험의 숲
- 그래프 시티 — 연결된 세상의 도시

6. 시스템 아키텍처 06



7. 관리자 대시보드 (시안) 07



8. AI 기반 콘텐츠 개선 프로세스 08

1 데이터 분석 — 정답률, 이탈 구간, 반복 행동 분석
2 개선안 제시 — AI가 개선 방향과 근거를 제안
3 관리자와 채팅 — 개선 방향 협의 수정 범위 확정
4 코드 수정 — AI가 브랜치에서 코드·콘텐츠 수정
5 자동 테스트 — 기능·정형성 검증 미리보기 빌드
6 관리자 승인 — 최종 검토 후 배포 클릭

AI 분석 보고
연결 리스트 포인터 연결 순서 문제의 정답률이 42%입니다. 학습자의 반복 재생 행동을 분석한 결과, 포인터 변경 순서를 더 명확히 보여줄 필요가 있습니다.
개선안: 애니메이션 세분화, 메모리 구조 표시, C 코드 하이라이트 추가

관리자
노드가 이동하는 것처럼 보이지 않게 수정해줘. C 코드도 한 줄씩 강조되도록 해.

AI 수정 결과
- 자료구조 데스트 통과
- C 코드 실행 검증 통과
- 앱 미리보기 빌드 성공

```
// 노드 삽입
newNode->next = current->next;
current->next = newNode;
```

9. 배포 구조 (Expo) 09

개발 브랜치 (AI 코드 수정) → 자동 빌드 & 테스트 (GitHub Actions): 단위 테스트 · 자료구조 검증 · 앱 빌드(Preview) → 관리자 승인: 미리보기 확인 배포 버튼 클릭 → Expo EAS: EAS Update (OTA 업데이트) · EAS Build & Submit (앱스토어 제출)

변경 유형별 배포 방식

변경 내용	배포 방법	앱스토어 심사
학습 설명, 퀴즈, 시나리오	콘텐츠 DB 게시	불필요
UI/애니메이션 수정	EAS Update (OTA)	일반적으로 불필요
네이티브 기능 추가	EAS Build + Submit	필요

10. 데이터 분석 & 개인화 학습 10

수집 데이터 예시

터치 좌표 (조작 패턴) · 학습 진행도 (단계별 체류 시간) · 퀴즈 결과 (정답률, 오답 유형) · 반복 행동 (재생, 다시 보기) · 앱 성능 (앱 시동 환경)

개인화 학습 추천

학습 데이터 분석 (Spark) → 학습자 프로필 (이해도, 취약 구간) → 맞춤 콘텐츠 추천 (난이도 조절, 복습)

- 기초 학습자 — 더 많은 시각적 설명과 기초 예제 제공
- 중급 학습자 — 문제 중심 학습과 다양한 시나리오 제공
- 고급 학습자 — 심화 문제와 C 코드 구현 실습 제공

11. 개발 로드맵 11

Phase 1 (1-3개월) — MVP 개발
- 연결 리스트 학습 콘텐츠
- 기본 캐릭터 애니메이션
- 학습 데이터 수집

Phase 2 (4-6개월) — 핵심 기능 확장
- 스택, 큐, 트리, 그래프 추가
- 관리자 대시보드
- AI 개선 제안 기능

Phase 3 (7-9개월) — AI 콘텐츠 개선
- 데이터 분석 고도화
- AI 코드 수정 에이전트
- 배포 시스템 구축

Phase 4 (10-12개월) — 정식 서비스
- 개인화 학습 추천
- 콘텐츠 자동 개선 루프
- 사용자 확대 및 고도화

12. 기대 효과 12

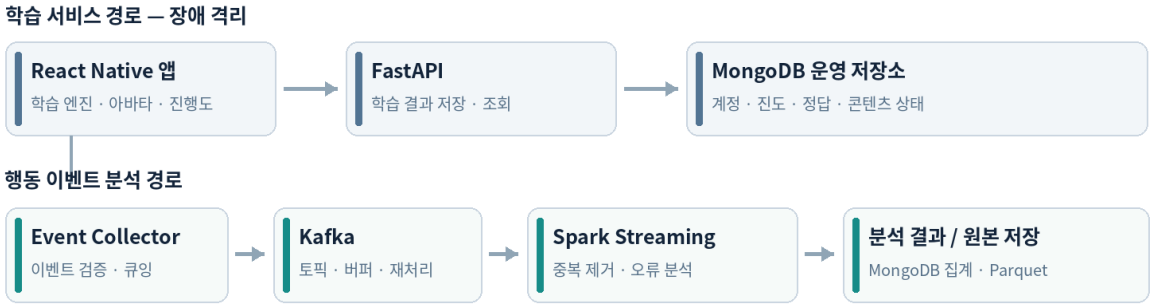
- 학습 효율 향상 — 시각적 이해와 실습을 통한 높은 학습 요과
- 지속적 콘텐츠 개선 — 데이터 기반 AI 개선으로 더 나은 학습 경험 제공
- 운영 효율화 — 관리자 업무 부담 감소, 콘텐츠 관리 용이
- 확장 가능성 — 다양한 프로그래밍 개념으로 확장 가능

※ 콘셉트 이미지의 일부 문자와 예시 수치는 구현 사양이나 검증된 운영 지표가 아닙니다.

NODEVERSE

아바타 기반 자료구조 시뮬레이션 및 학습 분석 플랫폼

C 언어로 이해하는 자료구조 × React Native 모바일 학습 × 데이터 기반 학습 개선



SERVICE CONCEPT / TECHNICAL PROPOSAL

기획 검토용 • Version 0.1 • 2026.10.22

0. 핵심 요약

- **제품의 본질:** 게임형 콘텐츠가 아닌, 캐릭터가 연산을 '행동으로 보여주는' 모바일 학습 시뮬레이터.
- **교육 목표:** 배열·포인터·연결 리스트·스택·큐에서 출발해 C 메모리 모델과 알고리즘 사고를 연결한다.
- **기술 방향:** React Native + TypeScript + Expo Development Build, Skia / Reanimated 기반 2D 행동 표현, FastAPI, MongoDB.
- **데이터 확장:** 앱 이벤트 → 수집 API → Kafka → Spark Structured Streaming → MongoDB 집계 및 오브젝트 스토리지 원본 보관.
- **제품 철학:** 오답·반복·힌트 사용 등의 기록만으로 이해도를 확정하지 않고, 사전·사후 문제 및 전이 문제 결과로 학습 효과를 검증한다.
- **개발 전략:** 먼저 학습 엔진과 1 개 월드의 정확성을 검증한 후, 4 개 월드 MVP → 이벤트 분석 → 맞춤 추천으로 확장한다.

1. 문제 정의와 프로젝트 목적

1.1 해결하려는 문제

- C 언어의 포인터·메모리·자료구조를 텍스트와 그림만으로 배우면 실제 상태 변화를 이해하기 어렵다.
- 기존 교육용 시각화는 노드나 화살표의 변화는 잘 보여주지만, 초급자가 '왜 이 행동이 필요한가'를 직관적으로 파악하기 어렵다.
- 일반적인 학습 앱은 정답률 위주로 진도를 제공하여 사용자가 어느 연산 단계에서 막혔는지 파악하기 어렵다.
- 학습 콘텐츠 개선이 강사의 추측이나 단편적 만족도에 의존하기 쉬우므로, 기능별 학습 데이터를 활용한 검증 체계가 필요하다.

1.2 목표와 비목표

목표: (1) 아바타 행동과 실제 연산 상태를 동기화해 오개념을 줄인다. (2) C 코드·메모리 뷰·캐릭터 행동을 하나의 단계로 보여준다. (3) 학습자가 조작한 이벤트를 통해 막힌 구간을 파악한다. (4) 검증 가능한 근거로 복습과 난이도를 추천한다.

비목표: 고사양 3D 오픈월드, 실시간 다중 사용자 게임, 초기 단계부터 대규모 생성형 AI 튜터 운영, 임의 C 코드의 모바일 단말 직접 실행, 단순 터치 횟수 기반의 이해도 단정.

2. 제품 콘셉트와 학습 경험

2.1 사용 경험의 기본 원칙

- **아바타는 선생님이 아니라 실행 주체:** 데이터 공을 들고 이동하고, 메모리 칸을 방문하며, 노드 사이의 연결을 바꾼다.
- **학습자가 실행을 제어:** 한 단계 실행, 뒤로 보기, 속도 조절, 정지, 다시 실행, 단계별 설명을 제공한다.
- **세 관점 동기화:** 캐릭터 월드 / 자료구조·메모리 뷰 / C 코드 하이라이트가 같은 이벤트 타임라인을 공유한다.
- **연출과 실제 계산을 분리:** 걷는 거리나 캐릭터의 애니메이션 시간이 실제 알고리즘 시간복잡도를 의미하지 않게 한다.
- **선행 개념 표시:** 배열 접근과 포인터 연산을 구분하고, 현재 삽입 위치를 알고 있는지 여부를 명시한다.

2.2 초기 월드 구성

월드	아바타 행동	핵심 개념	대표 미션
배열 실험실	공을 인덱스 칸에 배치·이동	연속 저장, 인덱스, 중간 삽입	2 번 칸에 15 를 넣기 위해 원소 이동
포인터·연결 리스트 숲	노드 생성 후 연결선을 바꿈	struct, malloc, next, NULL, free	10→30 사이에 20 연결

세계관 및 아바타 콘셉트

게임 클리어가 아니라 캐릭터가 자료구조 연산을 행동으로 시연하는 학습 경험



월드	아바타 행동	핵심 개념	대표 미션
스택 탑	가장 위에 올리고 위에서만 꺼냄	LIFO, push, pop	괄호 문자열 판별
큐 터미널	뒤에 넣고 앞에서 내보냄	FIFO, enqueue, dequeue	작업 요청 순서 처리
후속: 그래프 도시	거점 사이 간선 생성·탐색	정점, 간선, BFS, DFS	연결된 도시 방문 순서 탐색

2.3 대표 시나리오 — 연결 리스트 삽입

1. 학습자가 10→30 사이에 20 을 추가하도록 요청한다.
2. 학습 엔진이 '새 노드 메모리 할당' 이벤트를 생성하고, 아바타가 새 노드 20 을 만든다.
3. newNode->next = current->next 를 실행하며 20 의 연결을 30 으로 지정한다.
4. current->next = newNode 를 실행하며 10 의 연결을 20 으로 바꾼다.
5. 캐릭터, 메모리 구조, 코드의 현재 줄을 같은 타임라인으로 재생한다.
6. 연산 직후 '왜 연결 순서가 중요한가?' 확인 질문과 복습 미션을 제시한다.

정확성 가드레일: 연결할 위치의 선행 노드를 이미 알고 있으면 포인터 수정 자체는 O(1)이지만, 처음부터 위치를 탐색하면 O(n)이 추가될 수 있다. 시뮬레이션에서 두 상황을 분리해 가르친다.

3. 대상 사용자와 핵심 기능

3.1 사용자

유형	니즈	핵심 가치
자료구조 입문 학습자	C 포인터, 노드, 삽입·삭제가 어렵다	눈으로 보고 직접 단계별 실행
코딩테스트 준비생	어떤 자료구조를 선택할지 모르겠다	개념 → 연산 → 복합 문제 연결
교육 담당자	학습자가 정확히 어디서 막히는지 모른다	익명 집계·학습 병목 분석
서비스 운영자	앱 성능과 콘텐츠 이탈 원인을 모르겠다	품질·사용성 관찰 및 개선

3.2 기능 요구사항과 우선순위

ID	기능	우선순위	수용 기준 예시
F-01	2D 아바타와 노드 조작	P0	삽입·삭제 동작을 프레임 오류 없이 순차 표현
F-02	결정적 학습 엔진	P0	동일 상태·명령에서 동일 상태 전이 발생
F-03	코드·메모리·아바타 단계 동기화	P0	모든 연산 이벤트가 3 개 뷰에 동일하게 반영
F-04	학습 퀘스트와 정오답 판정	P0	학습 목표별 미션 제출 및 피드백
F-05	학습 기록·이어하기	P0	앱 종료·재진입 후 진행 상황 복구
F-06	세션·행동 이벤트 수집	P1	중복 이벤트 식별, 오프라인 재전송
F-07	학습 대시보드	P1	월드별 완료율, 오류 유형, 힌트 활용률 조회
F-08	규칙 기반 학습 추천	P1	오답 유형에 따른 복습 및 다음 미션 제안
F-09	C 코드 직접 입력·실행	P2	별도 격리 서버에서 제한 실행 가능
F-10	그래프 및 개인화 모델	P2	신규 월드 및 추천 평가 파이프라인 구현

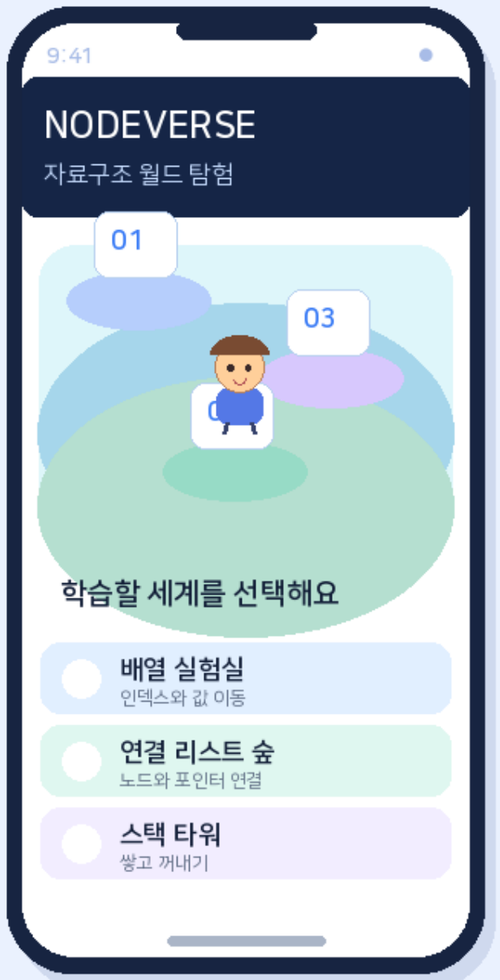
모바일 UX 시안 · React Native + Expo

01 월드 선택 / 02 연결 리스트 삽입 시뮬레이션과 C 코드 / 03 퀴즈 피드백

MOBILE PRODUCT CONCEPT

아바타 행동과 C 코드가 연결되는 학습 경험

월드 탐험 → 연결 리스트 연산 → 코드 이해·퀴즈



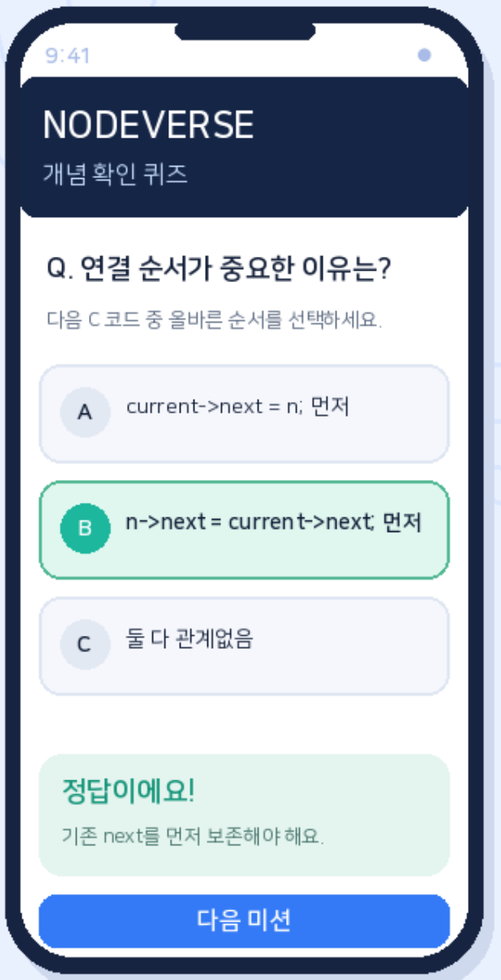
01 · 월드 선택

학습 목표와 진행도



02 · 연산 시뮬레이션

캐릭터 · 메모리 · C 코드



03 · 이해도 검증

오답 원인 및 피드백

시안 목적: 행동 · 메모리 · C 코드가 하나의 학습 단계에 동기화되는 경험

DESIGN CONCEPT · v0.2

4. 모바일 앱 기술 설계 — React Native

4.1 선택 이유

- React Native + TypeScript 로 iOS 와 Android 의 학습 UI 및 상태 관리를 공동 구현한다.
- React Native Skia 로 캐릭터, 공, 노드, 연결선 같은 2D 요소를 그린다.
- Reanimated 로 상태 전이에 따른 이동·크기·강조 애니메이션을 구현하고 Gesture Handler 로 탭·드래그를 받는다.
- Expo Router/Expo 모듈을 활용하되 확장성을 위해 **Expo Development Build** 를 기본 개발 환경으로 사용한다. Expo Go 만 전제한 설계는 피한다.
- 학습 엔진은 React 컴포넌트와 독립된 순수 TypeScript 모듈로 제작하여 단위 테스트와 재현 실험을 쉽게 한다.

4.2 권장 모듈 구성

계층	역할	후보 기술
화면·내비게이션	월드 선택, 강의, 결과, 계정	React Native, TypeScript, Expo Router
2D 렌더링	아바타·공·노드·간선 그리기	React Native Skia
모션·제스처	타임라인, 터치 조작, 드래그	Reanimated, Gesture Handler
학습 엔진	연산 검증, 상태 전이, 이벤트 생산	순수 TypeScript, 테스트 프레임워크
앱 데이터	인증, 진도, 네트워크, 캐시	React Query 계열, 로컬 저장소
관측 모듈	앱 버전·성능·학습 이벤트	별도 이벤트 큐, 필요 시 APM SDK

4.3 중요한 구현 경계

학습 엔진 ≠ 애니메이션 엔진. 학습 엔진이 ALLOCATE_NODE, SET_NEXT, MOVE_VALUE, COMPARE 같은 의미 있는 이벤트를 생성한다. 렌더러가 이 이벤트를 시각화하고, 이벤트 수집기는 행동 결과를 기록한다. 이렇게 해야 애니메이션을 수정해도 자료구조의 정답 판정이 달라지지 않는다.

C 코드의 학습과 C 코드 실행은 별개. 1 차 버전은 검증된 C 예제와 상태 전이를 동기화한다. 자유로운 C 편집·컴파일·실행은 2 차 이후 기능이며, 서버 격리 실행·시간 및 메모리 제한·네트워크 차단·감사를 갖춰야 한다.

5. 서비스 및 데이터 플랫폼 아키텍처

5.1 권장 구조

운영 경로: 모바일 앱 → FastAPI → MongoDB(계정, 학습 진도, 제출 결과). 이벤트 분석 장애가 학습 기능을 중단시키지 않도록 분리한다.

분석 경로: 모바일 앱(이벤트 큐) → Event Collector API → Kafka → Spark Structured Streaming → MongoDB(집계 지표) + 오브젝트 스토리지(정제·원본 이벤트). 이후 추천 서비스가 집계 결과를 조회한다.

관리 경로: 관리자 대시보드가 지표, 학습 콘텐츠 버전, 앱 품질을 조회하고 실험 결과를 기록한다.

5.2 왜 Kafka 다음 Spark 인가

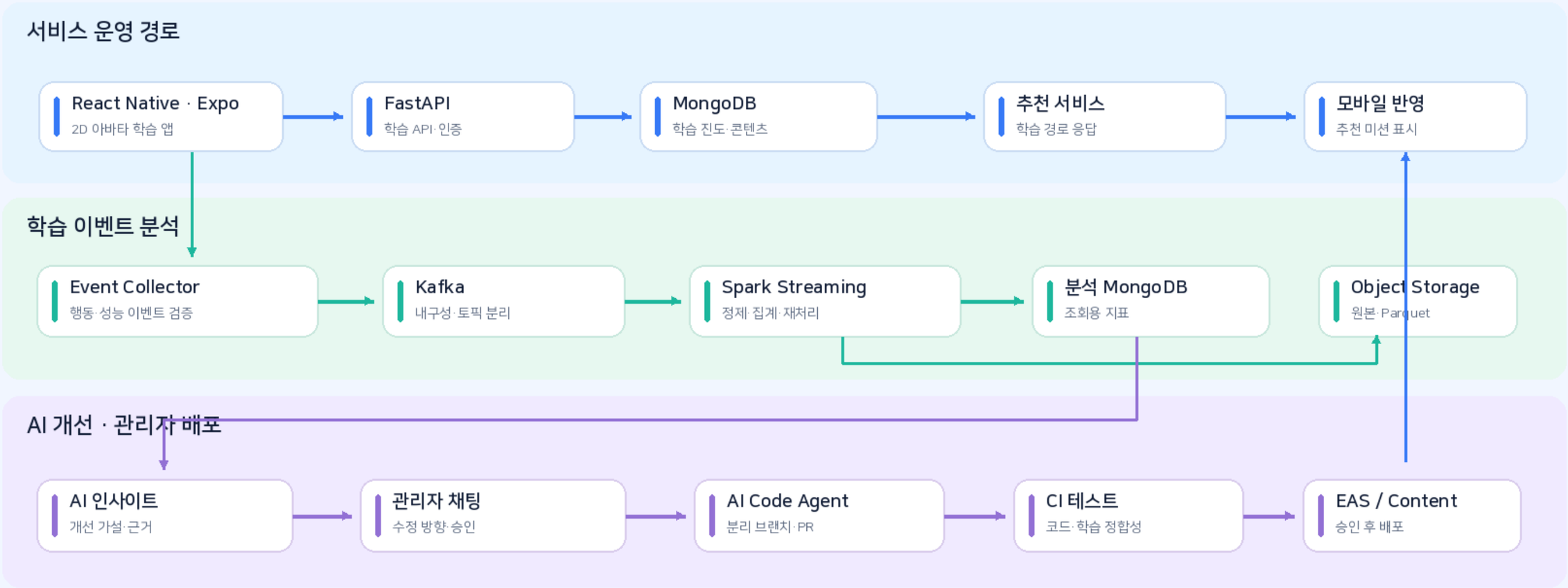
- Kafka 는 이벤트를 버퍼링하고 여러 소비자에게 전달하며 일정 기간 재처리할 여지를 제공한다.
- Spark Structured Streaming 은 Kafka 이벤트를 읽어 세션 집계, 유효성 검증, 중복 처리 및 오류 패턴 분석을 수행한다.
- MongoDB 는 운영 서비스의 현재 상태와 조회용 분석 결과를 저장한다. 대량 원본 이벤트의 장기 이력은 Parquet 등을 이용한 별도 저장소가 합리적이다.

시스템 아키텍처 · 운영 / 이벤트 분석 / AI 개선·배포

학습 서비스 API와 이벤트 분석 파이프라인을 분리하고, 승인형 AI 개발 절차를 별도로 운영

NODEVERSE · TECHNICAL ARCHITECTURE

운영 서비스 · 학습 이벤트 분석 · AI 개선/배포의 분리



운영 경로: 앱 → FastAPI → MongoDB | 분석 경로: Collector → Kafka → Spark → 분석 DB / Object Storage

모바일 성능 관측 아키텍처 · 학습 데이터와 연결

Expo 성능 지표, 크래시 진단, 자체 이벤트 수집을 분리하고 관리자 개선·배포 루프로 연결

React Native + Expo

학습 UI · 캐릭터 애니메이션

C 코드/메모리 동기화

화면·강의·콘텐츠 버전

성능 이벤트 최소 수집

USER EXPERIENCE

경로 A | 앱 성능

EAS Observe

시작·화면 렌더/TTI

Expo Observe 대시보드

EAS Build·OTA 버전별 비교

경로 B | 오류·크래시

Sentry

JS 오류·네이티브 크래시

Sentry / 스토어 진단

스택트레이스·릴리스

경로 C | 자체 이벤트

Event Collector API

학습 행동 + 표준화 성능 이벤트

Kafka → Spark → MongoDB

인증·유효성·샘플링·배치

공통 분석 키로 학습 지표와 성능 문제를 함께 해석

lesson_id | screen_id | app_version | update_id | device_class | session_id

예: 연결 리스트 문제의 이탈률 증가 → 콘텐츠 난이도 문제인지, 특정 Android 기기의 프레임 저하 때문인지 분리 검증

관리자 관측/분석

앱 품질 · 학습 성과 · 릴리스 비교

AI 개선안 도출

오답 유형·성능 회귀를 분리 분석

관리자 승인 및 수정

대화·PR·QA · 배포 승인

배포 후 검증

OTA / 앱 버전 추적

설계 주의: EAS Observe/Sentry 지표는 각 서비스에서 관측하고, 자체 Kafka 파이프라인과의 자동 동기화는 별도 연계 검증 후 확정.

- Kafka→Spark→MongoDB 구간이 전 구간 자동 exactly-once 인 것은 아니다. event_id 고유성, upsert, 체크포인트, 재처리 검증이 필요하다.

5.3 단계별 기술 도입

단계	운영 구성	의도
검증형 프로토타입	React Native + 학습 엔진 + 로컬 이벤트 로그	학습 경험과 설명 정확성 확인
MVP	FastAPI + MongoDB + 경량 수집 API	학습 진도·이벤트·기초 대시보드 운영
데이터 엔지니어링 확장	Kafka + Spark + Object Storage + MongoDB 집계	스트리밍·재처리·운영 지표 증명
적응형 학습 확장	추천 서비스 + 실험 관리 + 모델 관리	학습 효과 개선 검증

6. 이벤트 데이터 설계

6.1 무엇을 수집할 것인가

분류	예시 이벤트	분석 질문
학습	lesson_start, lesson_complete	어떤 월드에서 이탈하는가?
연산	operation_requested, operation_completed	어느 연산에서 반복 실패하는가?
이해 확인	quiz_submitted, hint_opened	어떤 오개념이 지속되는가?
UI 행동	node_drag_end, connection_attempt	잘못 연결하는 지점은 어디인가?
기술 품질	render_slow, api_failed, app_crash	기기·앱 버전별 문제는 무엇인가?

행동좌표 정의: GPS 위치가 아니라 앱 화면 내에서 조작한 상대 좌표를 말한다. 좌표는 전체 화면의 연속 이동 궤적 대신 source_node_id, target_slot_id, 조작 결과 같은 의미 단위로 우선 저장한다. 화면 크기 독립적인 좌표가 필요한 경우에만 0~1 정규화 좌표를 제한적으로 수집한다.

6.2 이벤트 예시

```
{
  "event_id": "uuid-v4",
  "event_type": "connection_attempt",
  "occurred_at": "2026-10-22T07:15:00+09:00",
  "anonymous_user_id": "hashed-user-id",
  "session_id": "session-id",
  "lesson_id": "linked-list-01",
  "content_version": "1.0.0",
  "app_version": "0.1.0",
  "source_node_id": "node-20",
  "target_node_id": "node-30",
  "outcome": "success",
  "duration_ms": 810,
  "coordinate": {"x_norm": 0.46, "y_norm": 0.31}
}
```

6.3 이벤트 정합성과 프라이버시

- 이벤트는 UUID 와 스키마 버전을 가진다. 서버에서 검증하고 타임스탬프를 보정한다.
- 모바일 오프라인 큐는 저장 기한·재시도 횟수·전송량을 제한하고, 재수신 이벤트는 event_id 로 중복 제거한다.
- 개인식별정보와 학습 행동 이벤트를 분리하고, 데이터 수집 목적·보관 기간·동의 및 철회 정책을 설계한다.
- 실수로 수집된 전체 터치 궤적, 화면 녹화, 입력 원문, 기기 식별자 등은 기본 수집 대상에서 제외한다.
- 어린이·청소년 대상 확대 시 별도의 개인정보 및 보호자 동의 요건을 검토한다.

7. 학습 분석과 개인화 추천

7.1 단계별 추천 전략

7. **초기 — 규칙 기반:** 특정 오류 유형이 반복되면 해당 개념을 설명하는 미션을 추천한다. 규칙은 실험 가설로 관리한다.
8. **중기 — 패턴 분석:** 월드·미션·개념별 정답률, 재시도, 힌트 의존도, 전이 문제 결과를 이용해 학습 군집을 분석한다.
9. **후기 — 예측 모델:** 충분한 품질의 레이블과 데이터가 확보된 경우에만 지식 추적 또는 추천 모델을 도입하고, 단순 규칙과 효과를 비교한다.

7.2 추천 시나리오

관측: 연결 리스트 삽입에서 next 변경 순서를 여러 번 틀림 → **가설:** 기존 연결 정보가 사라지는 위험을 이해하지 못했을 가능성 → **추천:** 노드 3 개로 연결 순서를 비교하는 짧은 재학습 → **검증:** 설명 없는 유사 문제에서 올바른 순서를 수행하는지 평가.

주의: 터치가 느리거나 반복 행동이 많다고 이해도가 낮다고 단정하지 않는다. 접근성·기기 성능·조작 실수 등을 통제해야 한다.

7.3 교육 지표와 서비스 지표 분리

교육 성과 지표	서비스 운영 지표
사전·사후 평가 점수 변화	앱 실행 성공률·크래시율
전이 문제 해결률	화면 전환 지연 및 렌더링 지연
1 주 후 재평가 정답률(선택)	API 오류율 및 지연 분포
개념별 오답 유형 개선	이벤트 전송 성공·유실·중복 비율
설명 없이 재수행 성공률	Kafka lag·Spark 처리 지연·DLQ 발생

8. MVP 와 단계별 로드맵

Phase 0 — 교육적 타당성 검증

- 연결 리스트 1 개 연산을 종이 시나리오 및 최소 애니메이션으로 구현한다.
- C 연산 순서와 메모리 상태를 검수하고 3~5 개의 오개념 질문을 구성한다.
- 입문 학습자 소수에게 사용성·이해도 사전 테스트를 진행한다.
- **통과 조건:** 사용자가 애니메이션 없이도 연결 순서를 설명할 수 있는지 확인한다.

Phase 1 — 학습 앱 MVP

- 배열·연결 리스트·스택·큐 4 개 월드의 핵심 연산을 구현한다.
- 단계 실행, 되감아 보기, 퀘스트, 정답 피드백, 학습 진도 저장을 제공한다.
- Expo Development Build 기반 Android 우선 또는 iOS 우선으로 실제 기기에서 검증한다.
- 이 단계에서는 Kafka 와 Spark 없이도 학습 데이터를 소량 수집해 검증할 수 있다.

Phase 2 — 데이터 엔지니어링 확장

- 이벤트 스키마 고정, Collector 도입, Kafka 토픽과 DLQ 구성.
- Spark Structured Streaming 정제·집계, 중복 제거, 체크포인트 및 재처리 검증.
- MongoDB 학습 분석 컬렉션과 운영 대시보드 구축.
- 결과 신뢰성을 위해 원본 이벤트 수·집계 수·누락 수가 일치하는지 검증한다.

관리자 콘솔 시안 · 분석과 AI 개선 협업

학습 지표 관찰 → AI 개선안 채팅 → 코드 수정 검토 → 배포 승인으로 연결되는 운영 화면

N NODEVERSE

ADMIN CONSOLE

- 대시보드
- 학습 분석
- AI 개선 제안
- 콘텐츠 버전
- 배포 관리
- 운영 모니터링

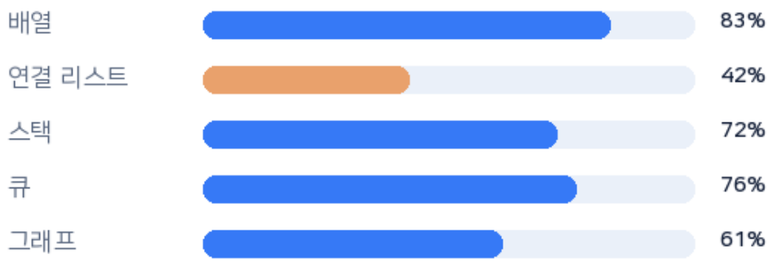
학습 콘텐츠 개선 센터

학습 병목 탐지 → AI 제안 → 관리자 협의 → 코드 검증 → 승인 배포

주간 활성 학습자	연결 리스트 정답률	AI 제안 대기	배포 안정성
2,480명	42%	7건	99.2%
+12.4%	개선 후보	검토 필요	7일 기준

개념별 정확도

최근 4주 · 예시 데이터



AI 개선안 #LL-021

검토 필요

문제: next 포인터 순서 오개념

가설: 노드 이동 연출이 참조 변경과 혼동됨

제안: 캐릭터 연출 + 코드 하이라이트 수정

근거 확인 | AI와 개선안 협의

관리자 ↔ AI 협의

AI · 포인터 변경 순서를 설명하는 2개 장면을 추가하겠습니다.

관리자 · 먼저 코드와 메모리 뷰를 동시에 표시해줘.

검증 및 배포

- C 연산 단위 테스트
- 애니메이션 스냅샷
- EAS Preview 배포

관리자 최종 배포

대시보드에 표시된 성과 수치는 실제 운영 데이터가 아닌 시안용 예시입니다.

관리자 콘솔 확장 시안 · 모바일 성능 모니터링

앱/OTA 버전 단위의 성능 회귀·크래시·학습 이탈을 함께 관찰하는 관리자 화면 예시

NODEVERSE ADMIN

- 개요 대시보드
- 학습 성과
- 앱 성능 ●
- 오류/크래시
- AI 개선 제안
- 배포/롤백

모바일 성능 모니터링

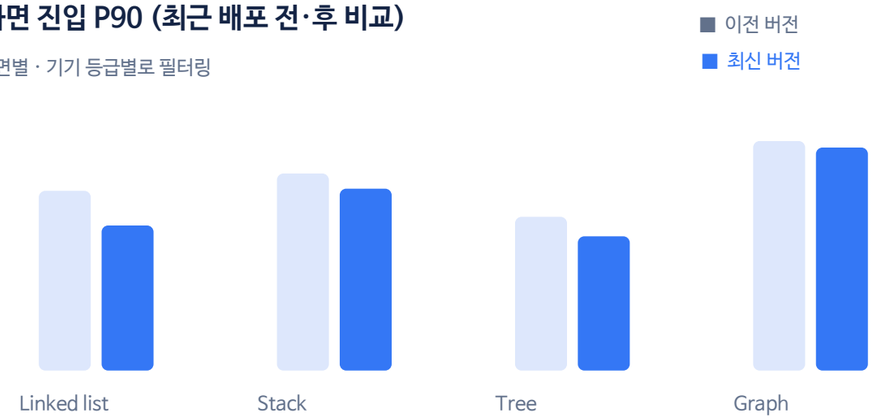
Release | 1.2.0 / OTA-19

예시 대시보드 · 운영 데이터 아님 · 동일한 앱/OTA 버전 기준 비교

Cold start P90	학습 화면 TTI P90	Crash-free sessions	Frozen frame 비율
2.8s	1.4s	99.6%	0.8%
목표 ≤ 3.0s	목표 ≤ 1.5s	목표 ≥ 99.5%	추세/기기별 감시

학습 화면 진입 P90 (최근 배포 전·후 비교)

ms | 화면별 · 기기 등급별로 필터링



릴리스 리스크 알림

WATCH

OTA-19 적용 기기 중
일부 저사양 Android 기기에서
연결 리스트 애니메이션 지연 증가

원인 분석 | 배포 일시중지

학습 분석과 성능 분석의 결합 예시

포인터 학습 화면의 이탈률이 상승했더라도, 콘텐츠 설명이 어려워졌다고 단정하지 않는다.

screen_id + lesson_id + update_id를 이용해 화면 렌더링 지연·API 실패·기기 성능을 먼저 확인한다.

성능 문제이면 기술 개선 PR, 개념 문제이면 학습 콘텐츠 개선안을 AI가 각각 생성한다.

※ 수치는 UI 구성을 설명하기 위한 가상 예시. 목표값도 초기 제안이며 실제 측정값으로 조정한다.

Phase 3 — 적응형 학습 고도화

- 규칙 기반 추천, 문제 유형 분류, 추천 노출·선택·결과 로그 구축.
- 사전·사후 평가와 대조군 또는 순차적 실험으로 개선 효과 판단.
- 그래프 월드·힙 월드 및 C 코드 실행 기능을 선택적으로 확장한다.

9. 품질, 보안, 운영 설계

9.1 데이터 파이프라인

- FastAPI 입력 검증, rate limiting, 인증·권한 분리, 민감 데이터 마스킹.
- Kafka 토픽 정책 및 보관 기간, 파티션 키 설계, consumer lag 경보.
- Spark 체크포인트, 이벤트 유효성 검사, 스키마 변경 관리, 중복·유실 테스트.
- MongoDB 조회 인덱스, TTL, 집계 컬렉션과 원본 데이터 분리.

9.2 C 코드 실행기(후속)

- 신뢰할 수 없는 C 코드는 모바일에서 실행하지 않는다.
- 독립 샌드박스에서 비특권 실행, CPU·메모리·실행 시간 제한, 외부 네트워크 차단, 출력 크기 제한.
- 플랫폼 보안 검토 전에는 사용자가 자유롭게 작성한 C 코드를 서버에서 실행하도록 공개하지 않는다.

9.3 성능·접근성

- 저사양 기기 기준 애니메이션 일관성, 배터리 영향, 메모리 사용량을 테스트한다.
- 색상만으로 의미를 전달하지 않고 텍스트·패턴·선 굵기로 상태를 함께 나타낸다.
- 모션 최소화, 애니메이션 속도 조절, 화면 읽기 지원을 고려한다.
- 성능 이슈를 학습 실패로 잘못 분류하지 않도록 이벤트에 기기·버전·지연 상태를 요약 기록한다.

10. 주요 리스크와 의사결정 기준

리스크	영향	대응
캐릭터 연출이 개념을 왜곡	학습 효과 저하	연산 상태 머신을 단일 정답 소스로 사용
2D 그래픽 최적화 부담	저사양 기기 프레임 저하	Skia 장면 수 제한 및 실제 기기 성능 측정
지나친 좌표 수집	개인정보·저장비용 문제	의미 이벤트 우선, 좌표 최소화·보관 정책
Kafka·Spark 과도 설계	MVP 지연·복잡성 증가	데이터 규모와 목표에 맞춰 Phase 2 도입
개인화의 근거 부족	오추천·편향	규칙 기반 시작, 사전·사후 평가 검증
C 실행기 보안	임의 코드 실행 취약점	후속 기능으로 분리, 강한 격리 보장

11. 성공 기준과 다음 의사결정

첫 번째 검증 질문: 아바타가 노드를 옮기는 장면을 본 학습자가 포인터 연결 순서를 이전보다 더 정확히 설명하고 재현하는가?

제품 검증 지표: 특정 개념의 전후 평가 개선, 미션 완료율, 도움말 없이 재수행률, 실제 모바일 조작 안정성. 지표의 목표치는 초기 사용자 테스트 결과를 기준으로 설정한다.

데이터 엔지니어링 검증 지표: 이벤트 누락·중복·재전송 시나리오 통과, Kafka 소비 지연 관측, Spark 체크포인트 복구, MongoDB 집계 정합성, 스키마 버전 변경 대응.

우선 확정할 5 가지 결정: (1) 첫 월드는 연결 리스트, (2) 2D 캐릭터 표현 방식, (3) C 코드 동기화 범위, (4) 저장할 최소 행동 이벤트, (5) 앱 MVP 와 데이터 플랫폼의 릴리스 분리.

12. 참고 기술 문서

- Expo FAQ 및 Development Build: https://docs.expo.dev/faq/ ; https://docs.expo.dev/develop/development-builds/faq/
- React Native Skia 애니메이션: https://wcandillon.github.io/react-native-skia/v2/docs/animations/animations/
- Apache Spark Structured Streaming + Kafka: https://spark.apache.org/docs/latest/structured-streaming-kafka-integration.html
- MongoDB Spark Streaming Connector: https://www.mongodb.com/docs/spark-connector/current/streaming-mode/streaming-write/
- 위 기술은 현재의 구현 후보이며, 사용 라이브러리의 세부 호환 버전은 착수 시 잠금 파일과 공식 호환표를 기준으로 재확인한다.

13. AI 기반 학습 콘텐츠 개선 운영 설계

핵심 운영 원칙: AI는 개선 근거를 제시하고 소스코드를 수정하지만, 수정 착수와 운영 배포는 각각 관리자의 명시적 승인으로 통제한다.

13.1 운영 흐름

단계	동작	통제·증적
01 데이터 관측	Kafka → Spark로 학습 오답·재시도·화면 성능 지표 분석	표본 수, 분석 기간, 콘텐츠 버전, 앱 버전
02 AI 개선안	학습 실패 원인 가설과 콘텐츠·시나리오 수정 방향 생성	근거 지표와 반론·대체 설명 포함
03 관리자 채팅	개선 범위·구현 방식 협의, 수정 요청 반영	전체 대화/결정 로그, 변경 요청 스냅샷
04 개발 승인	관리자 승인 후 제한된 AI Code Agent 실행	수정 허용 경로, 작업 범위, 승인자 기록
05 코드 변경	격리 브랜치에 수정, PR 생성, 테스트 실행	Diff, 테스트 결과, 위험 변경 목록
06 검수·배포	관리자가 최종 변경 내용 확인 후 배포 클릭	배포 대상·릴리스 ID·변경 이력
07 효과 검증	개선 버전의 학습 효과 및 오류율을 관찰	사전·사후·전이 평가, A/B 또는 단계적 배포

13.2 AI와 관리자 대화 예시

AI: 연결 리스트 노드 삽입의 특정 문항 오답률이 높고, 포인터 순서 단계의 재시도가 많습니다. 기존 설명이 참조 변경과 노드 이동을 혼동하게 만들었을 가능성이 있습니다.

관리자: 노드를 옮기는 애니메이션을 줄이고 next 포인터 변경을 단계별로 보여줘. C 코드 줄 강조도 추가해.

AI: 변경 파일 목록과 테스트 계획을 제안합니다. 승인하면 격리 브랜치에서 수정하고 PR 및 미리보기를 생성합니다.

관리자: 개발 승인. 테스트 통과 뒤 차이점과 미리보기를 다시 확인하겠습니다.

13.3 자동 수정 허용 범위

초기 단계에서는 강의 설명, 퀴즈, 애니메이션 시나리오, 일부 React Native 화면 컴포넌트로 제한한다. 인증·결제·데이터 삭제·보안 설정·네이티브 권한·CI 배포 권한의 변경은 별도 강화 승인 대상으로 분리한다.

14. AI 코드 에이전트와 배포 통제

14.1 코드 수정 파이프라인

관리자 개발 승인 → 범위가 제한된 작업 티켓 생성 → 임시 작업공간/브랜치 → AI 코드 수정 → 정적 검사 및 테스트 → PR → 미리보기 빌드 → 관리자 최종 승인 → 릴리스 워크플로 실행.

필수 검증 항목

검증 영역	필수 기준
교육 정합성	C 코드와 자료구조 상태 전이 일치, 포인터 연결 순서 및 불변식 검증
동작 회귀	같은 상태·명령에 동일한 결과, 삽입/삭제 경계조건 테스트
모바일 품질	앱 부팅·화면 전환·성능 회귀·접근성 및 주요 기기 스모크 테스트
데이터 이벤트	스키마 유효성, event_id, 학습 버전 필드, 개인정보 최소 수집
보안/배포	비밀값 미노출, 권한 제한, 관리 승인 상태 확인, 롤백 가능성

14.2 Expo 기반 배포 분기

변경 대상	배포 전략	유의 사항
설명·퀴즈·시나리오 JSON	콘텐츠 서버 버전 게시	앱 재배포 불필요. 콘텐츠 롤백 가능하게 설계
JS/TS·스타일·이미지	EAS Update (OTA)	기존 바이너리 runtimeVersion과 호환, 스토어 정책 준수
네이티브 모듈·권한·설정	EAS Build + Submit	신규 바이너리·스토어 검토 필요
고위험 기능 변경	별도 릴리스 심의	관리자 추가 승인·점진적 공개

14.3 릴리스 상태 모델

INSIGHT_FOUND → PROPOSED → IN_REVIEW → APPROVED_FOR_DEV → PR_READY → QA_PASSED → APPROVED_FOR_RELEASE → RELEASED → MONITORING. 테스트 실패 또는 기각 시 REJECTED/REWORK로 돌아가며, 배포 후 문제가 발생하면 ROLLED_BACK 상태로 전환한다.

배포 클릭은 무조건적인 즉시 업데이트를 뜻하지 않는다. 콘텐츠 게시, OTA 전파, 새 앱 심사/설치의 결과와 소요 조건은 서로 다르며 실제 반영 상태를 콘솔에 표시해야 한다.

15. 학습 효과 검증과 개선 루프

15.1 버전 단위의 추적 가능성

분석부터 배포까지 **lesson_id / content_version / app_version / runtimeVersion / experiment_id / deployment_id**로 연결한다. 이벤트와 배포 이력이 버전별로 분리되어야 어떤 변경이 어떤 결과를 만들었는지 추적할 수 있다.

15.2 개선 판단의 근거

분석 결과	성급한 해석	검증해야 할 다른 원인
애니메이션 재생 횟수 증가	이해 부족	관심·재학습·느린 기기·접근성
포인터 문제 정답률 저하	설명 오류	문제 난이도·선행 지식·평가 문항 오류
학습 완료율 상승	학습 효과 개선	평가 난이도, 표본 구성, 실제 전이 문제 성과
업데이트 후 이탈 증가	새 UI 문제	네트워크/렌더링 오류, 사용자 환경 변경

15.3 측정 계획

연결 리스트 삽입 강의를 파일럿 대상으로 삼는다. 개선 전후 학습 집단의 사전지식과 문제 난이도를 고려하여 비교한다. 단기에는 연결 순서 설명 정확도·전이 문제 정답률을, 장기에는 복습 유지와 성능 회귀 여부를 본다.

핵심 KPI

① 개념별 사전·사후 평가 변화 ② 학습 지원 없이 재수행 성공률 ③ 새로운 문제로의 전이 성공률 ④ 미션 이탈률·힌트 사용률 ⑤ 앱 크래시/프레임 저하 ⑥ 이벤트 누락·중복 및 스트리밍 처리 지연.

15.4 초기 MVP 구현 원칙

처음부터 AI가 앱 코드를 자동 수정하게 만들기보다, **연결 리스트 한 강의에서 개선 제안 → 관리자 대화 → 콘텐츠 시나리오 변경 → 검증 → 배포**를 먼저 종단간 시연한다. 이후 코드 에이전트와 EAS Update 배포를 점진적으로 연결한다.

개인정보·안전 고려

터치 전체 경로·GPS·화면 녹화는 기본적으로 수집하지 않는다. 상호작용 대상과 결과 중심 이벤트를 저장하고,
보관기간·삭제·동의·접근통제·감사로그를 정의한다. AI 생성 코드에는 운영 DB 쓰기 및 배포 자격증명 직접 접근을 허용하지 않는다.

본 문서의 화면 수치와 가상 학습 분석 사례는 설명 목적의 예시이다. 실제 성능·학습 효과를 입증하는 측정 결과가 아니다.

16. 모바일 앱 성능 모니터링 체계

학습 성과의 변화와 앱 성능 문제를 구분하여 분석한다. 예를 들어 연결 리스트 강의의 이탈률이 상승할 때, 교육 내용의 난이도 문제인지 화면 렌더링 지연 때문인지 검증한 뒤 개선 방향을 정한다.

16.1 모니터링 수집 범위

분류	주요 지표 / 수집 시점	활용
앱 시작	Cold/warm launch, 첫 렌더, TTI (P50/P90)	버전별 초기 진입 성능
학습 화면	월드/강의/퀴즈 화면 렌더, 화면 이동 TTI	취약 화면·사용 기기 비교
애니메이션	연산 시작~표현 완료 시간, 느린/정지 프레임	Skia/Reanimated 성능 회귀
안정성	Crash-free sessions, JS 예외, 네이티브 크래시	운영 품질·릴리스 차단
통신	학습 API P95, 타임아웃, 오류율, 재시도	서버/네트워크 원인 분리
배포	app/build/update ID, 다운로드 시간, 적용률	OTA 변경 전후 성능 비교

16.2 수집 기술 분담

EAS Observe: Expo 앱의 실제 사용자 환경에서 시작·렌더링·화면 이동·업데이트 관련 성능 지표를 관측하고 빌드/OTA 버전별 추이를 비교한다. 지원 여부와 SDK/빌드 조건은 도입 시점의 공식 문서로 확인한다.

Sentry: JavaScript 오류, 네이티브 크래시, 스택트레이스 및 릴리스 태그를 추적한다. Expo EAS와 연동해 배포 후 회귀 원인을 찾는다.

자체 Telemetry SDK + Collector: 학습 맥락이 필요한 사용자 정의 성능 이벤트만 최소 수집하여 Kafka → Spark → MongoDB/객체 스토리지로 처리한다. EAS Observe 또는 Sentry의 원본 데이터를 Kafka로 자동 전송하는 구조는 연동 지원이 확인되기 전까지 가정하지 않는다.

16.3 학습 이벤트와 성능 이벤트의 연관 분석 키

필드	설명	비고
event_id, occurred_at	중복 제거용 식별자, UTC 이벤트 시각	버퍼 지연 고려
session_id, anonymous_user_id	학습 세션 및 가명 사용자 식별	원문 개인정보 제외
lesson_id, screen_id, action_id	학습 맥락·화면·연산/명령	좌표보다 의미 중심
app_version, build_id, update_id	네이티브 및 OTA 릴리스 식별	전/후 비교 필수
platform, device_class	OS·단말 성능 등급	원시 기기 ID 금지

17. 성능 기반 AI 개선 및 운영 프로세스

17.1 개선안 자동 도출과 관리자 승인

① 성능·학습 집계 생성 → ② 임계치/기기군별 이상 징후 탐지 → ③ AI가 가능한 원인과 반례를 함께 제시 → ④ 관리자가 AI 채팅으로 확인 및 수정 범위 승인 → ⑤ 코드 에이전트가 제한된 브랜치에 패치/PR 생성 → ⑥ 성능 회귀·학습 정합성 테스트 → ⑦ 관리자 배포 클릭 → ⑧ 릴리스 모니터링 및 점진 확대/중단.

17.2 초기 성능 목표 (실제 운영 기준 확정 전, 예시)

관측 항목	초기 목표 예시	조치 규칙 예시
Cold start P90	3.0초 이하	연속 악화 시 원인 분석
학습 화면 TTI P90	1.5초 이하	기기 등급별로 분리 분석
Crash-free sessions	99.5% 이상	급락 시 배포 확대 중단
학습 동작 응답 P95	150ms 이하	연산별 UI/JS 경로 재점검
느린/정지 프레임	이전 버전 대비 악화 없음	실제 기기군별 기준 수립

목표값은 제품 요구사항을 논의하기 위한 초기 예시다. 기기/OS/네트워크가 서로 다른 실제 운영 데이터로 재산정해야 하며, 한 가지 종합 점수보다 지표별 원인을 보는 편이 적절하다.

17.3 관리자 모니터링 화면 필수 기능

앱 안정성 요약, 기간 및 Android/iOS 필터, 화면별 성능, 앱/OTA 버전 비교, 문제 강의 식별, 오류 상세 링크, 알림 이력, AI 개선안, 코드 변경 PR, 테스트 결과, 단계적 배포 및 롤백 상태를 하나의 흐름으로 연결한다.

17.4 검증·배포 안전장치

개선 전/후 비교는 동일한 기기군·앱 버전 계열·학습 난이도 조건에서 수행한다. 배포 전 릴리스 빌드에서 측정하고, EAS Update 진행 중 크래시율/화면 TTI 악화가 감지되면 추가 노출을 중단하도록 관리자에게 제안한다. 네이티브 코드 변경은 EAS Build 및 스토어 배포 흐름을 따른다.

17.5 개인정보·비용·운영 고려

전체 터치 궤적, 사용자가 입력한 C 코드 전문, 정밀 위치, 화면 녹화는 기본 수집하지 않는다. 터치 좌표가 꼭 필요하면 짧은 보관 기간·표본 수집·비식별화·동의 여부를 별도로 설계한다. 오프라인에서는 유실 방지를 위한 제한된 배치 버퍼와 재시도를 사용하고, 이벤트 수·샘플링 비율·보관 기간으로 비용을 관리한다.

공식 기술 확인 자료 (2026년 10월 확인)

- https://docs.expo.dev/eas/observe/ (Expo EAS Observe)
- https://docs.expo.dev/guides/using-sentry/ (Expo + Sentry)
- https://reactnative.dev/docs/performance.html (React Native Performance)
- https://docs.expo.dev/monitoring/services/ (Expo Monitoring Services)